Representación de memoria y datos primitivos en C: Un análisis del modelo mental del programador novato

Pablo Riofrio – pablo.riofrio@unl.edu.ec

https://orcid.org/0009-0007-3023-9951

***Resumen*— Los programadores novatos suelen enfrentar dificultades para asimilar conceptos fundamentales como el tamaño en bytes, los rangos de representación y los límites de precisión de los tipos de datos primitivos. Este ensayo analiza los errores más frecuentes cometidos por principiantes al manipular enteros, reales, caracteres y más en C.**

Abstract*— Novice programmers often struggle to grasp fundamental concepts such as size in bytes, representation ranges, and precision limits of primitive data types. This essay examines the most common errors made by beginners when working with integers, floating-point numbers, characters, and other data types in C.*

***Palabras clave*— Novato, memoria, datos primitivos, variables.**

I. Introducción

La unidad de memoria más pequeña en un ordenador se denomina bit (del inglés binary digit). Puede tomar únicamente dos posibles valores: 0 o 1. En ocasiones, debido a la relación intrínseca con los valores en las señales eléctricas en circuitos digitales, se dice que un bit está bajo o alto, o bien desconectado o conectado. Como puede verse, no es posible almacenar mucha información en un solo bit. Sin embargo, un ordenador posee cantidades ingentes de ellos, por lo que podría decirse que los bits son los bloques básicos con los que se construye la memoria del ordenador [1].

El byte, compuesto por ocho bits, es una unidad de memoria más útil. Puesto que cada bit puede tomar el valor 0 o 1, en un byte pueden representarse hasta 256 combinaciones de ceros y unos. Con estas combinaciones pueden representarse, por ejemplo, los enteros entre 0 y 255, un conjunto de caracteres, etc. [1].

La unidad natural de memoria para un ordenador es la palabra. Los ordenadores de sobremesa actuales, por ejemplo, suelen trabajar con palabras de 32 o 64 bits. En grandes ordenadores, el tamaño de la palabra puede ser mucho mayor, pero siempre formada por un número de bits, potencia de 2. En cualquier caso, los ordenadores encadenan dos o más palabras de memoria con el fin de poder almacenar datos complejos y, en general, de mayor tamaño [1].

II. Tamaño de los Datos Primitivos

Existen varios tipos de variables, y cada uno corresponde a un tamaño máximo de un número, carácter, incluso una verdad. Mientras mayor sea el número que pueda admitir, mayor espacio en la memoria ocupará [2].

Char— Las variables de tipo char tienen un tamaño en memoria de 1 byte con unos valores posibles de -128 a 127 o 0 a 255. Este tipo de variable guarda caracteres del código ASCII [2].

Int— Las variables de tipo int tienen un tamaño en memoria de 2 bytes con valores posibles de -32,768 a 32,767 o -2,147,483,648 a 2,147,483,647. Este tipo de variable guarda números enteros (sin punto decimal) [2].

Float— Las variables de tipo float tienen un tamaño en memoria de 4 bytes con valores posibles de - $3{,}4 \cdot 10^{38}$ a $3{,}4 \cdot 10^{38}$. Este tipo de variable guarda números decimales con precisión de 6 dígitos [2].

Short— Las variables de tipo short tienen un tamaño de 2 bytes con valores posibles de -32,768 a 32,767. Este tipo de variable guarda números enteros con menor capacidad de almacenamiento y rango que la variable int [2].

Double— Las variables de tipo double tienen un tamaño de 8 bytes con valores posibles desde – $1{,}79 \cdot 10^{308}$ a $1{,}79 \cdot 10^{308}$. Este tipo de variable es igual que float pero

con mayor precisión (hasta 15 dígitos decimales) y mayor rango [2].

Long— Este tipo de variable tiene un tamaño de memoria de 4 bytes y tiene valores posibles de -2,147,483,648 a 2,147,483,647. Guarda números enteros con mayor capacidad de almacenamiento que int [2].

Long double— Este tipo de variable tiene un tamaño de 10 bytes, siendo la variable de más peso. Tiene valores posibles de 3.4E-4932 a 1.1E+4932. Tiene una precisión mucho mayor en calculo decimal, incluso mayor que doublé (hasta 19 dígitos decimales) [2].

III. Rangos de Representación y Límites

El lenguaje C es un lenguaje tipado. Esto quiere decir que toda variable debe tener asociado un tipo de dato de los definidos en las librerías del lenguaje. Cada tipo de dato significa un tamaño en memoria y un rango de valores. En el caso de C los tipos básicos que dependen de la arquitectura del sistema son [2]:

Tipos y tamaño de datos en C dependiendo del OS [2].

C Basic Data Types	32-bit CPU		64-bit CPU	
	Size (bytes)	Range	Size (bytes)	Range
char	1	-128 to 127	1	-128 to 127
short	2	-32,768 to 32,767	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647	4	-2,147,483,648 to 2,147,483,647
long	4	-2,147,483,648 to 2,147,483,647	8	-9,223,372,036,854,775,808-9,223,372,036,854,775,807
long long	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807
float	4	3.4E +/- 38	4	3.4E +/- 38
double	8	1.7E +/- 308	8	1.7E +/- 308

Por lo que al declarar una variable «numero» de tipo entero que ocupa 4 bytes, el OS y su gestor buscarán un espacio mayor o igual a 4 bytes desasignados y se asociarán al identificador número. De esta forma todo valor guardado en número o leído de su posición en memoria queda asociado a la variable hasta que termine la ejecución del programa y se desasigne [2].

Por lo tanto, el desconocimiento de los límites de representación es una de las principales fuentes de errores lógicos en programadores novatos. Frecuentemente, se asume de manera errónea que un tipo int puede almacenar cualquier magnitud numérica, lo que inevitablemente conduce a un desbordamiento aritmético cuando el valor supera la capacidad máxima de sus bytes asignados [2].

IV. Conclusiones

El desarrollo en C exige abandonar la falsa premisa de que la memoria y la precisión numérica son ilimitadas. Un programador profesional entiende que cada declaración de variable está atada a restricciones de hardware irrefutables; por ejemplo, la drástica diferencia al almacenar un entero, cuyo límite superior en una arquitectura de 32 bits es de 2147483647, frente a los reducidos 32767 de un sistema de 16 bits. Ignorar esto y las limitaciones de precisión entre tipos como float y double es la raíz de errores lógicos más críticos. Por lo tanto, dominar los tipos de datos primitivos y sus modificadores no es una mera regla sintáctica, sino el cimiento indispensable para construir software seguro, eficiente y verdaderamente portable [2].

REFERENCIAS

[1] M. A. Peña Basurto y J. M. Cela Espín, *Introducción a la programación en C*. Barcelona: Edicions UPC, 2000.

[2] R. Pulgarín G., J. O. Herrera M., y J. E. Gutiérrez P., *Introducción a la Programación en C*. Colombia: ELIZCOM S.A.S, 2018.